NORTHSTAR TECHNOLOGIES PVT. LTD.

Backend Development Guidelines

Engineering practices for designing, building, testing, and maintaining reliable backend services

Document ID	ENG-GDE-007
Document Type	GUIDE
Access Scope	TEAM_ONLY
Department	Backend Engineering
Version	1.0
Effective Date	04 November 2026
Owner	Backend Engineering Team

INTERNAL TEAM GUIDANCE

This is a fictional demonstration document prepared for the OnBoardIQ platform. It is intended for training, document-ingestion, and retrieval testing; it is not a legally approved corporate standard.

1. Purpose and Scope

These guidelines establish a consistent baseline for backend software development at Northstar Technologies Pvt. Ltd. They apply to services, APIs, background workers, integrations, and data-access components maintained by the Backend Engineering team.

- **Build for maintainability:** prefer clear, modular designs over unnecessary abstraction or tightly coupled components.
- **Protect data by default:** apply authentication, authorization, validation, and least-privilege access at service boundaries.
- **Design for failure:** assume networks, dependencies, queues, and external services can fail or respond slowly.
- **Make behaviour observable:** provide structured logs, useful metrics, traces where appropriate, and actionable alerts.
- **Automate quality:** use repeatable tests and CI checks before changes are merged or released.

Audience: backend engineers, technical leads, reviewers, and authorized engineering administrators. Other teams should use this guide only if they have been granted access through the document access policy.

2. Engineering Principles

All backend work should follow these principles:

- **Single responsibility:** modules and services should have clear ownership and focused responsibilities.
- **Explicit contracts:** API inputs, outputs, error formats, event schemas, and configuration should be documented.
- **Backward compatibility:** avoid breaking existing clients and consumers without a coordinated migration plan.
- **Secure defaults:** deny access unless it is explicitly authorized; never rely on frontend checks as the security boundary.
- **Operational readiness:** consider deployment, rollback, monitoring, backup, and incident response during design—not after release.
- **Pragmatism:** choose the simplest design that satisfies functional, security, reliability, and scale requirements.

3. Service Architecture and Code Organization

Services should be organized around well-defined business capabilities. Keep transport, business logic, and persistence concerns separated so that changes in one layer do not unnecessarily affect the others.

Recommended logical boundaries

Layer	Responsibility	Guideline
API / Transport	Routes, controllers, request parsing, response mapping	Keep controllers thin; delegate business decisions to services.
Application / Service	Use cases and orchestration	Coordinate domain operations and transactions.
Domain	Core rules, entities, invariants	Avoid coupling core rules to HTTP frameworks or database clients.
Persistence / Repository	Database queries and mapping	Centralize data access and avoid duplicated query logic.
Infrastructure	Queues, email, storage, external APIs	Wrap external systems behind interfaces or focused adapters.

Repository conventions

- Use consistent names for files, classes, functions, routes, database objects, and environment variables.
- Group code by feature or domain when the service grows; avoid oversized catch-all folders.

- Keep configuration separate from source code and validate required configuration during application startup.
- Do not commit generated build output, local secrets, temporary files, or machine-specific configuration.

4. API Design Standards

HTTP APIs should be predictable, discoverable, and consistent across services.

Area	Standard
Resource naming	Use nouns for resource paths, lowercase words, and consistent pluralization (for example, /api/v1/employees).
HTTP methods	Use GET for reads, POST for creation or commands, PUT for full replacement, PATCH for partial updates, and DELETE for removal where appropriate.
Status codes	Return meaningful HTTP status codes. Do not return 200 for failed operations.
Validation	Validate type, format, length, range, and business constraints on the server.
Pagination	Use a documented pagination approach for collection endpoints; enforce sensible maximum page sizes.
Filtering and sorting	Allow only documented fields and validate requested sort directions.
Versioning	Use a consistent versioning strategy and communicate deprecation before removing an API.
Idempotency	For retry-prone operations, consider idempotency keys or another duplicate-prevention mechanism.

Request and response conventions

- Use a consistent response envelope where it improves client integration; do not mix incompatible response shapes across endpoints.
- Return stable machine-readable error codes with safe, human-readable messages.
- Never expose stack traces, SQL errors, internal hostnames, secrets, or sensitive implementation details to clients.
- Document required fields, optional fields, nullability, constraints, and examples in an API specification such as OpenAPI.
- Set request size limits and appropriate timeouts. Reject malformed or unsupported content types.

5. Authentication, Authorization, and Application Security

Security controls must be enforced on the server for every protected operation. Authentication confirms identity; authorization determines whether that identity may perform the requested action.

- Use approved authentication mechanisms and secure token or session handling. Validate issuer, audience, expiry, signature, and relevant claims where applicable.
- Enforce role- and resource-level authorization in backend services. Check ownership and tenant/team boundaries for every relevant data operation.
- Apply least privilege to service accounts, database users, cloud identities, and third-party integrations.
- Store passwords only with an approved adaptive password-hashing algorithm. Never store plaintext passwords.
- Use TLS for service communication where supported and encrypt sensitive data at rest according to company requirements.
- Protect against injection, broken access control, insecure deserialization, SSRF, path traversal, and other common application risks.
- Use parameterized database queries or safe ORM query APIs; never concatenate untrusted input into SQL.

- Apply rate limits and abuse controls to sensitive or expensive endpoints.
- Keep dependencies patched and review security advisories for critical libraries.

Secrets management

- Never hardcode credentials, API keys, signing keys, or database passwords in source code.
- Use an approved secrets manager or protected deployment environment variables.
- Rotate credentials after suspected exposure and avoid printing secrets in logs, traces, test fixtures, or error messages.

6. Database and Data Access

Database design should protect data integrity and support predictable performance.

- Define primary keys, foreign keys, uniqueness constraints, and check constraints for important invariants.
- Use transactions for operations that must succeed or fail as a unit; keep transaction boundaries short.
- Create indexes based on measured query patterns. Review query plans for slow or high-volume operations.
- Avoid N+1 query patterns and unbounded queries. Fetch only the fields and rows required.
- Use migration scripts for schema changes. Migrations must be reviewed, repeatable, and safe for the deployment sequence.
- For large tables, plan backward-compatible expand-and-contract migrations rather than risky one-step changes.
- Define retention, archival, deletion, and backup expectations for data handled by the service.
- Do not log sensitive personal data or full records when identifiers or masked values are sufficient.

Before deploying a schema change, confirm whether old and new application versions can operate safely during a rolling deployment.

7. Error Handling and Resilience

Failures should be handled deliberately and surfaced with enough context for diagnosis without disclosing sensitive information.

- Use consistent error categories: validation, authentication, authorization, not found, conflict, rate limit, dependency failure, and unexpected server error.
- Do not silently swallow exceptions. Handle expected failures explicitly and allow unexpected failures to be recorded by centralized error handling.
- Set connection and request timeouts for outbound calls. Avoid indefinite waits.
- Use bounded retries with exponential backoff and jitter only for transient failures and operations that are safe to retry.
- Use circuit breakers or bulkheads for unstable dependencies when justified by service criticality.
- For asynchronous workflows, define retry limits, dead-letter handling, idempotent consumers, and a process for replaying failed messages.
- Return partial results only when the API contract explicitly supports partial success.
- Provide graceful shutdown so in-flight requests and message processing can finish or be safely requeued.

8. Background Jobs, Queues, and Event-Driven Workflows

Queue-based processing should be designed for duplicate delivery, delayed processing, and out-of-order events unless the chosen platform guarantees otherwise.

- Define event names, schema versions, required fields, producer ownership, and consumer expectations.
- Assume at-least-once delivery unless stronger guarantees are explicitly documented; make consumers idempotent.

- Acknowledge messages only after successful processing or durable handoff.
- Use retry policies with backoff and route repeatedly failing messages to a dead-letter queue.
- Include correlation IDs and event IDs to support tracing and duplicate detection.
- Monitor queue depth, oldest-message age, processing duration, retry count, and dead-letter volume.
- Avoid placing secrets or unnecessary personal information in event payloads.

9. Logging, Monitoring, and Observability

Production services must provide sufficient telemetry to understand health, performance, and failures.

- Use structured logs with consistent fields such as timestamp, severity, service, environment, request ID, and correlation ID.
- Log meaningful lifecycle events and errors; avoid excessive debug logging in production.
- Never log passwords, access tokens, session cookies, private keys, or sensitive personal information.
- Track request rate, error rate, latency, saturation, dependency health, and business-critical workflow metrics.
- Propagate trace and correlation context across service boundaries where supported.
- Create actionable alerts with an owner, severity, and runbook or troubleshooting guidance.
- Ensure health endpoints distinguish basic process liveness from readiness to serve traffic.

10. Testing and Quality Assurance

Testing should provide confidence at multiple levels and should run consistently in local development and CI.

Test type	Expected use
Unit tests	Verify isolated functions, domain rules, validators, and edge cases.
Integration tests	Verify database, cache, queue, and external adapter behaviour using controlled test dependencies.
API tests	Verify routes, authentication, authorization, validation, status codes, and response contracts.
Contract tests	Verify compatibility between service producers and consumers where independently deployed.
End-to-end tests	Verify critical user or business workflows across relevant components.
Performance tests	Measure throughput, latency, resource use, and behaviour under expected or peak load.

- Test success, failure, boundary, and permission-denied scenarios.
- Use deterministic test data and isolate tests from production systems.
- Mock external dependencies where appropriate, but retain integration tests for important real interactions.
- Fix flaky tests rather than repeatedly rerunning them without investigation.
- Do not reduce test coverage or disable security checks merely to make a pipeline pass.

11. Code Review and Git Workflow

- Create focused branches and pull requests with descriptive names and linked work items where applicable.
- Keep pull requests small enough to review. Separate unrelated refactoring from feature changes.
- A pull request description should explain the problem, solution, important design decisions, testing performed, and deployment considerations.
- Review correctness, security, readability, performance, error handling, observability, and test coverage.

- Do not approve your own change where an independent review is required by the team process.
- Resolve review comments with code changes or a documented technical discussion.
- Do not commit secrets, personal data, local environment files, or unrelated generated artifacts.
- Use the team's agreed branching, commit-message, and release conventions; document exceptions.

12. CI/CD and Release Management

The delivery pipeline should make builds and deployments repeatable, auditable, and recoverable.

- Run formatting, linting, unit tests, security/dependency checks, and build validation in CI.
- Build immutable artifacts and promote the same artifact through environments where practical.
- Keep environment-specific configuration outside the artifact and validate it before startup.
- Use staged or controlled rollouts for high-risk changes when supported.
- Define health checks and post-deployment verification for critical services.
- Maintain a rollback or forward-fix plan, including database compatibility considerations.
- Record release version, change summary, deployment owner, and relevant approvals.
- Do not deploy directly to production from an unreviewed local workstation.

13. Performance and Scalability

- Establish service-level expectations for latency, throughput, availability, and resource consumption.
- Measure before optimizing. Use profiling, query plans, load tests, and production telemetry to locate bottlenecks.
- Use caching only with a clear invalidation strategy, acceptable staleness, and protection against cache stampedes.
- Set connection-pool limits and concurrency controls to protect downstream systems.
- Avoid unbounded in-memory collections, synchronous work in request paths when asynchronous processing is more appropriate, and unnecessarily large payloads.
- Use pagination, streaming, batching, or asynchronous processing for large workloads.
- Plan capacity based on expected growth and test behaviour under realistic load.

14. Documentation and Ownership

- Document service purpose, owners, dependencies, runtime configuration, data stores, and deployment process.
- Maintain API specifications and event schemas alongside the code where practical.
- Provide a README with local setup, test commands, required environment variables (names only), and troubleshooting steps.
- Document important architecture decisions, including context, alternatives, trade-offs, and consequences.
- Maintain operational runbooks for common alerts, recovery steps, and escalation paths.
- Assign a clear service owner and review ownership when teams or responsibilities change.

15. Development Workflow Checklist

Use this checklist before requesting review or promoting a backend change:

Check	Expected evidence
Requirements	Acceptance criteria and edge cases are understood.

Check	Expected evidence
Design	API, data, event, and dependency changes are documented where needed.
Security	Authentication, authorization, validation, secrets, and data exposure have been reviewed.
Implementation	Code follows project conventions and handles expected failures.
Tests	Relevant unit, integration, API, and regression tests pass.
Observability	Logs, metrics, traces, and alerts are adequate for the change.
Operations	Configuration, migrations, rollout, and rollback are considered.
Review	Pull request has clear context and required approvals.

16. Roles and Responsibilities

Role	Responsibility
Backend Engineer	Implement, test, document, and support backend changes in accordance with these guidelines.
Reviewer / Senior Engineer	Review design and code quality, identify risks, and provide actionable feedback.
Backend Team Lead	Maintain technical direction, review significant design changes, and coordinate service ownership.
Engineering Administrator	Support approved engineering governance and access administration; does not replace technical review.

Questions about interpretation should be raised with the Backend Engineering Team Lead. Security incidents or suspected credential exposure must be reported immediately through the company's designated security incident channel.

17. Exceptions and Maintenance

A deviation from these guidelines should have a documented reason, an identified owner, an assessment of security and operational risk, and an agreed review or expiry date. Exceptions affecting authentication, authorization, sensitive data, or production reliability require appropriate engineering and security review.

The Backend Engineering Team should review this guide periodically and update it when architecture, technology, security requirements, or engineering practices materially change.

Document control: ENG-GDE-007 | Version 1.0 | Effective 04 November 2026 | Access Scope: TEAM_ONLY | Department: Backend Engineering

Demo notice: Northstar Technologies Pvt. Ltd. is a fictional company used for demonstration and OnBoardIQ document-ingestion testing. This guide is illustrative and does not constitute legal, regulatory, or formally approved corporate policy.